

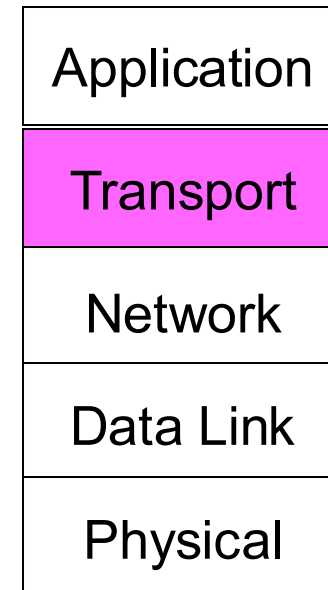
# Transport Layer (1)

COMP90007 Internet Technologies

Lecturer: Adel N. Toosi

# Layered Network

| Week | Topic                 |
|------|-----------------------|
| 1    | Introduction          |
| 2    | Physical Layer        |
| 3    | Data Link Layer       |
| 4    | Medium Access Control |
| 5    | Network Layer         |
| 6    | Network Layer         |
| 7    | Transport Layer       |
| 8    | Transport Layer       |
| 9    | Application Layer     |
|      | Non-teaching period   |
| 10   | Application Layer     |
| 11   | Network Security      |
| 12   | Review                |



# Outline

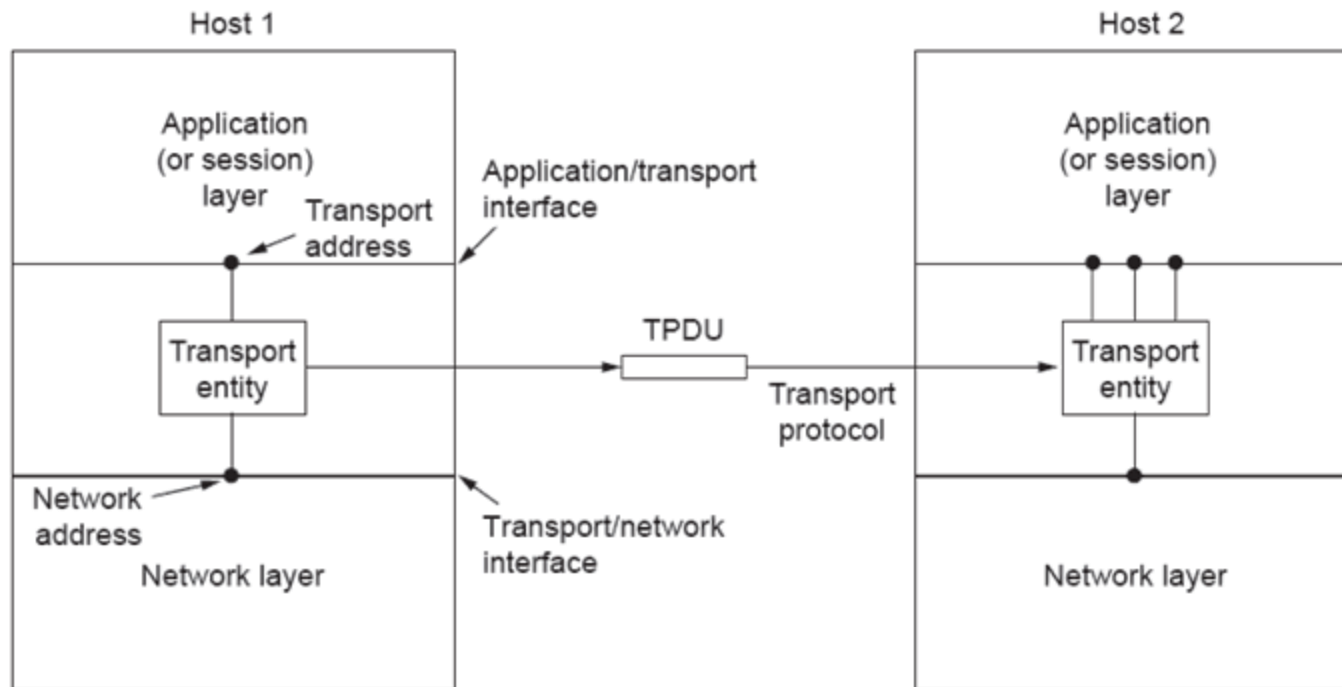
- Transport Layer Service
- Elements of Transport Protocols
- Programming using Sockets
- Transport Layer Protocols: TCP and UDP
- Quality of Service

# Transport Layer Function

- Provide **efficient, reliable & cost-effective** data transmission service to the *processes* in the application layer, independent of physical or data link layers.
- Calls services provided by the network layer to achieve this.

# Transport Layer Services

- **Transport Layer Services** provide interfaces between the Application Layer and the Network Layer



Relationship between network, transport and application layers

# Transport Layer Services

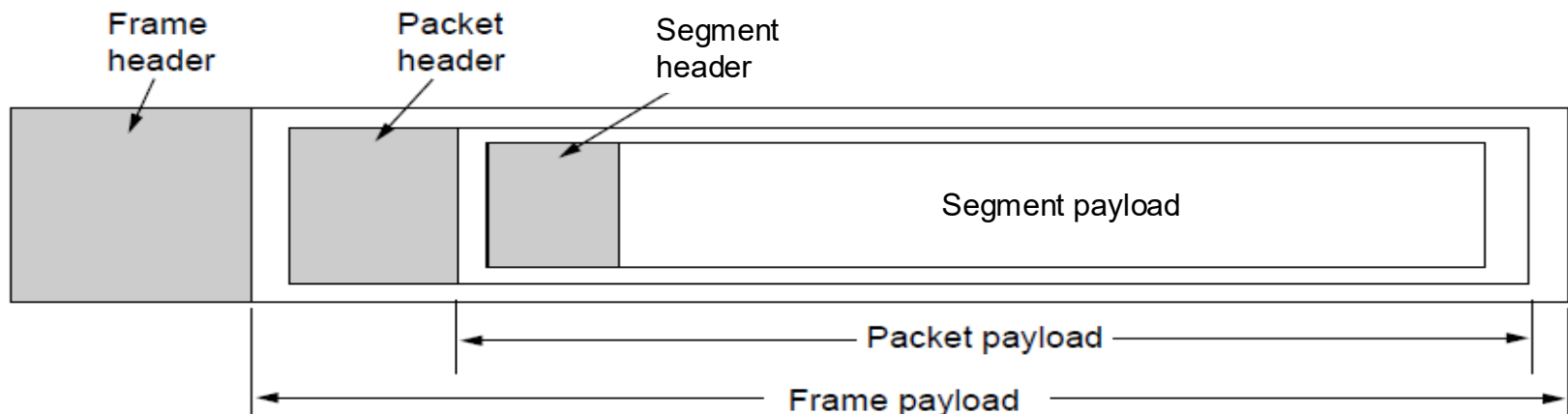
- **Transport Entities** can exist in multiple locations:
  - OS kernel
  - System library (library package bound into network applications)

Not so much...

- User process
- Network interface card

# Transport Layer Encapsulation

- Abstract representation of messages sent to and from transport entities
  - Transport Protocol Data Unit (TPDU) = segment
- Encapsulation of segments to packets and to frames



# Transport Layer Services

- Transport layer offers **connectionless** (e.g., UDP) in addition to **connection-oriented** (e.g., TCP) services to applications
- If Transport and Network layers are so similar, why are there two layers?
  - Transport layer code runs entirely on hosts.  
Network layer code runs almost entirely on routers.  
Users have no real control over the network layer.
  - Transport layer: can improve QoS, fix *reliability* problems caused by the network layer (e.g., delayed, lost or duplicated packets).



# Transport Layer Services

- Transport Layer occupies a key position in the layer hierarchy because it clearly delineates
  - **providers** of data transmissions services
    - at the network, data link, and physical layers
  - **users** of reliable data transmission services
    - at the application layer
- Users commonly access connection-oriented transport services for a **reliable service** on top of an unreliable network

# Features of a Simple Transport Layer

- Abstraction and primitives provide a simpler API for application developers independent of network layer

| Primitive  | Meaning                                    |
|------------|--------------------------------------------|
| LISTEN     | Block waiting for an incoming connection   |
| CONNECT    | Establish a connection with a waiting peer |
| RECEIVE    | Block waiting for an incoming message      |
| SEND       | Send a message to the peer                 |
| DISCONNECT | Terminate a connection                     |

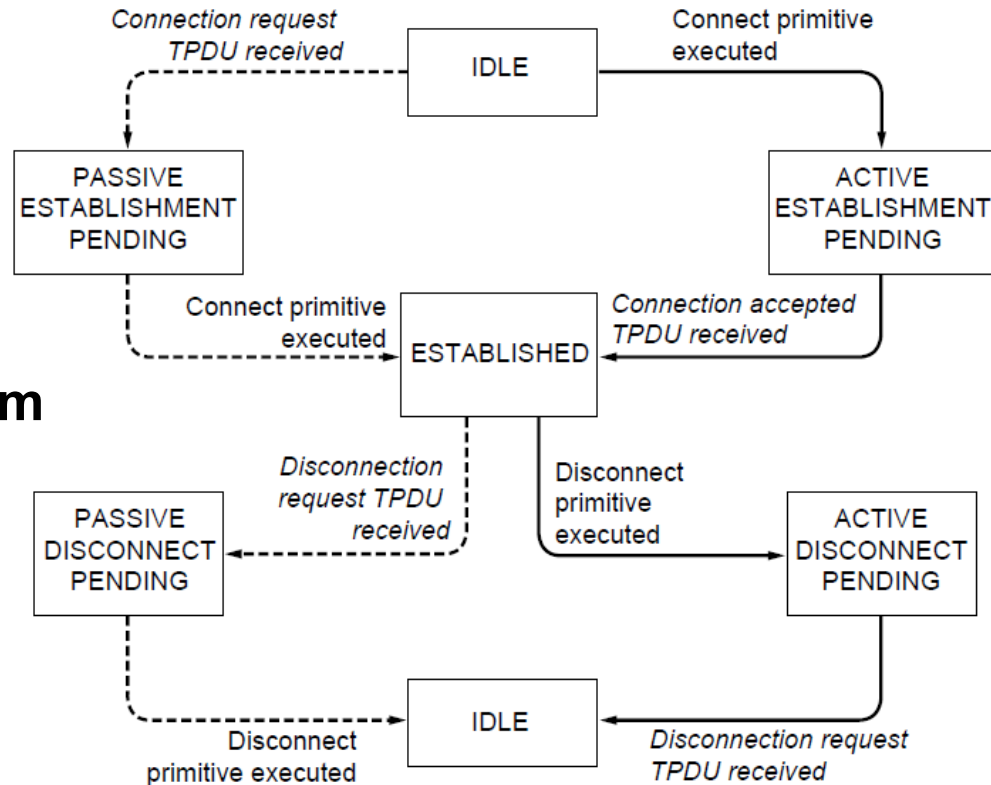
# Transport Service Primitives/Segments

- Primitives that applications might call to transport data for a simple **connection-oriented service**
  - Server executes LISTEN
  - Client executes CONNECT
    - Sends CONNECTION REQUEST segment to Server
    - Receives CONNECTION ACCEPTED segment
  - Data exchanged using SEND and RECEIVE
  - Either party executes DISCONNECT

| Primitive  | Segment sent       | Meaning                                    |
|------------|--------------------|--------------------------------------------|
| LISTEN     | (none)             | Block until some process tries to connect  |
| CONNECT    | CONNECTION REQ.    | Actively attempt to establish a connection |
| SEND       | DATA               | Send information                           |
| RECEIVE    | (none)             | Block until a DATA packet arrives          |
| DISCONNECT | DISCONNECTION REQ. | This side wants to release the connection  |

# Simple Connection Illustrated

## State Diagram



- Solid lines (**right**) show **client** state sequence
- Dashed lines (**left**) show **server** state sequence
- Transitions in italics are due to segment arrivals

# Example: Your First Network (Pseudo)Code

```
Socket A_Socket = createSocket("TCP");
```

```
connect(A_Socket, 128.255.16.0, 80);
```

```
send(A_Socket, "My first message!");
```

```
disconnect(A_Socket);
```

*... there is also a server component for this client  
that runs on another host...*

# Elements of Transport Protocols

- Connection establishment
- Connection release
- Addressing

# Connection Establishment

- Challenges: packets may be delayed, duplicated or lost
  - ❑ congested networks may delay acknowledgements
  - ❑ incurring repeated multiple transmissions
  - ❑ any of which may not arrive at all or out of sequence – delayed duplicates
  - ❑ applications degenerate with such congestion

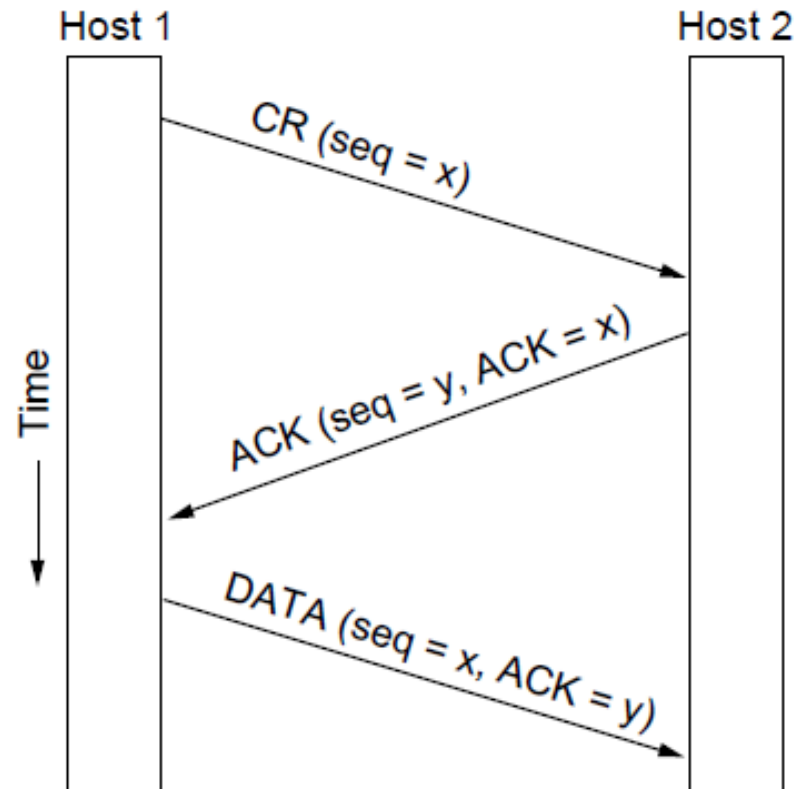
# Reliable Connection Establishment

- Goal: ensure reliability even though packets may be delayed, duplicated, lost, or corrupted
  - Don't treat an old or duplicate packet as new
  - Use repeat requests and checksums for loss or corruption
- Approach:
  - Don't reuse **sequence numbers** within maximum segment lifetime
  - Use a sequence number space large enough that it will not wrap, even when sending at full rate
  - **Three-way handshake** for establishing connection



# Three Way Handshake

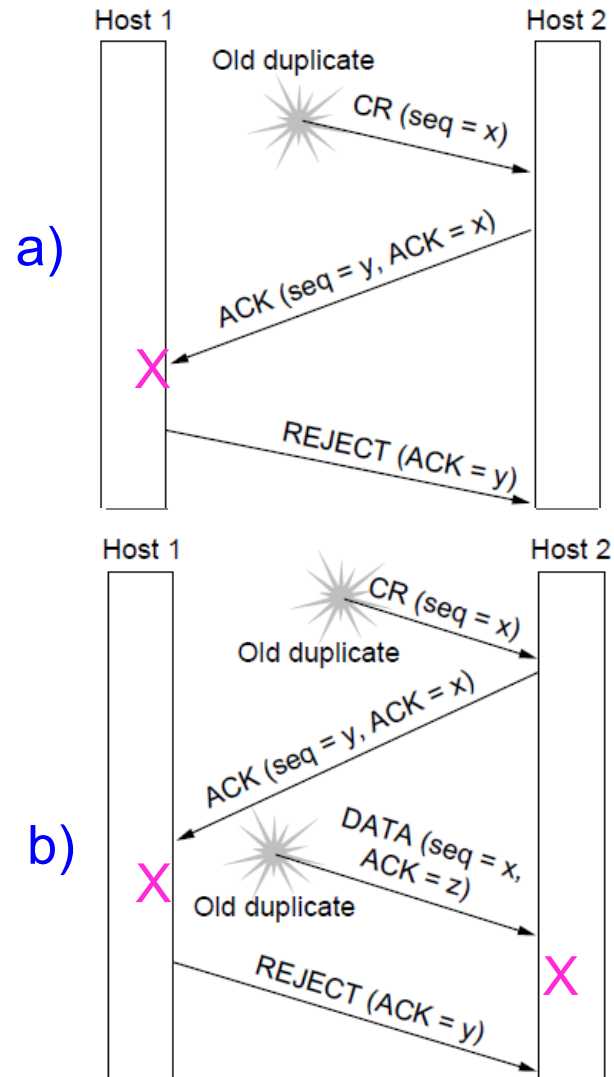
- Three-way handshake used for initial packet
  - Since no state from previous connection
  - Both hosts contribute fresh seq. numbers
  - CR = Connect Request



# Three Way Handshake

- Three-way handshake protects against error cases:

- a) Duplicate CR. Spurious ACK does not connect.
- b) Duplicate CR and DATA. Duplicate DATA will be rejected (wrong ACK).



# Connection Release

## ■ **Asymmetric Disconnection**

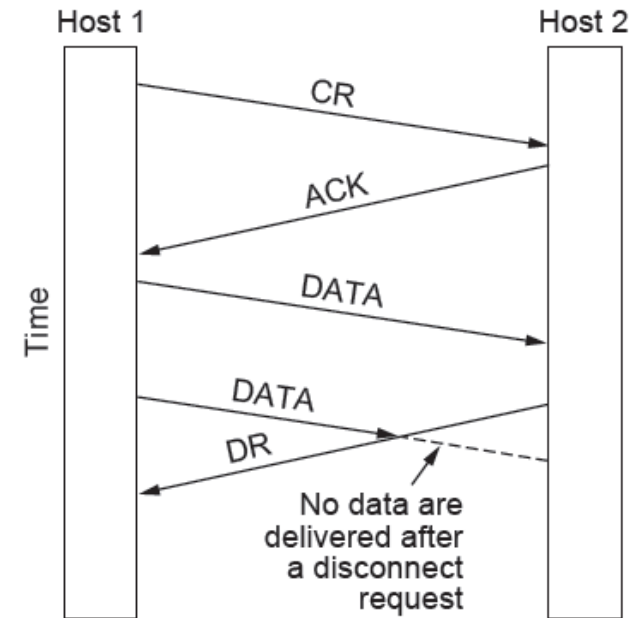
- Either party can issue a DISCONNECT, which results in DISCONNECT TPDU and transmission end in both directions

## ■ **Symmetric Disconnection**

- Both parties issue DISCONNECT, closing only one direction at a time – allows flexibility to remain in receive mode

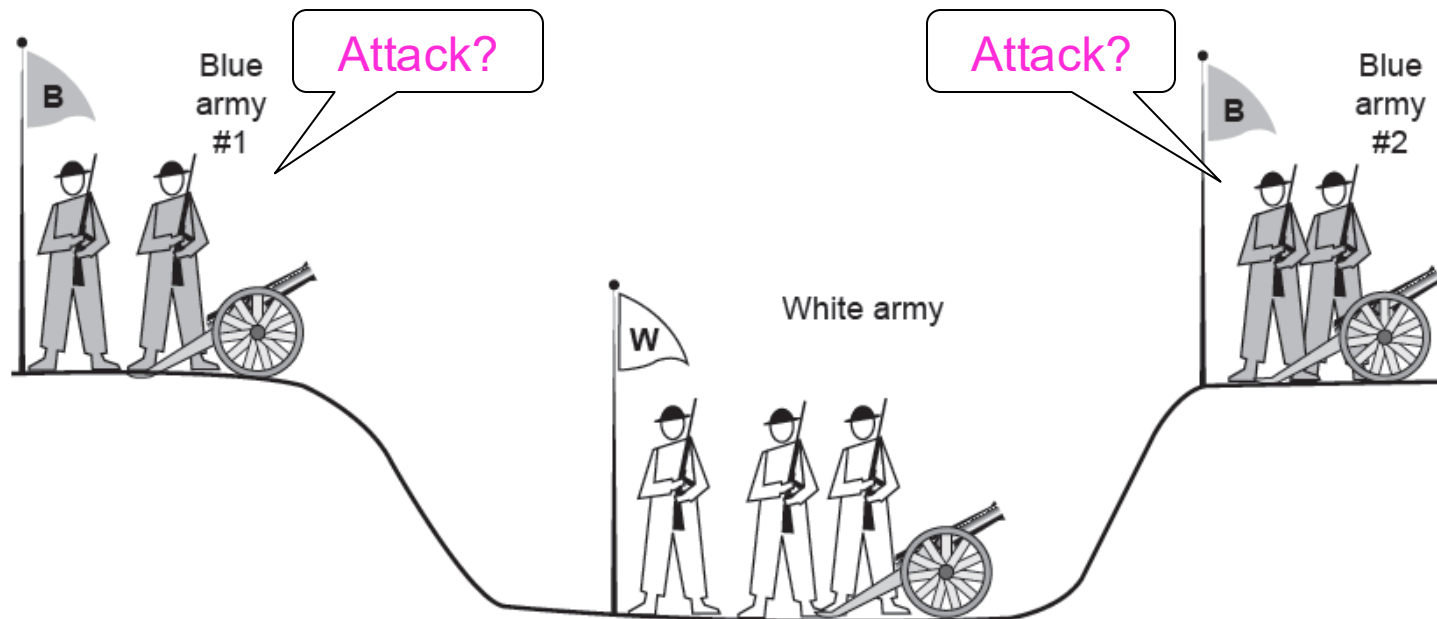
# Connection Release

- Asymmetric vs. Symmetric connection release
  - **Asymmetric** release may result in data loss hence symmetric release is more attractive
  - **Symmetric** release works well where each process has a fixed amount of data to transmit and knows it has been sent



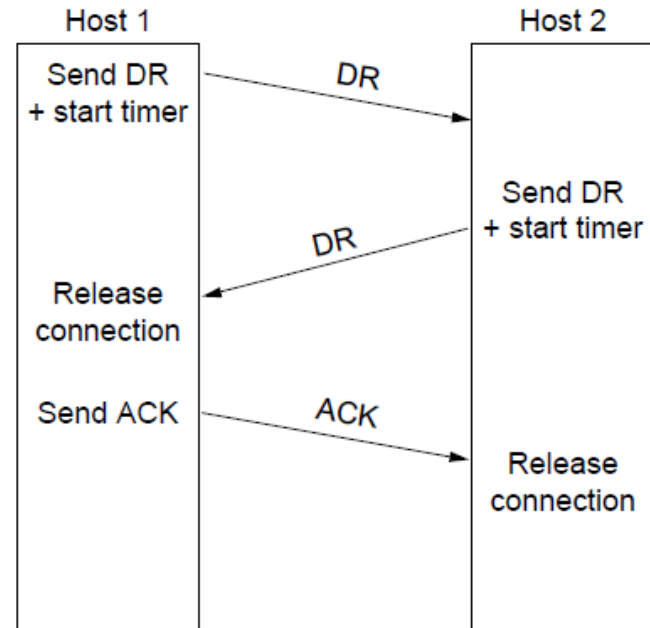
# Connection Release Problem

- How do we decide the importance of the last message? Is it essential or not?
- No protocol exists which can resolve this ambiguity
  - Two-army problem shows pitfall of agreement



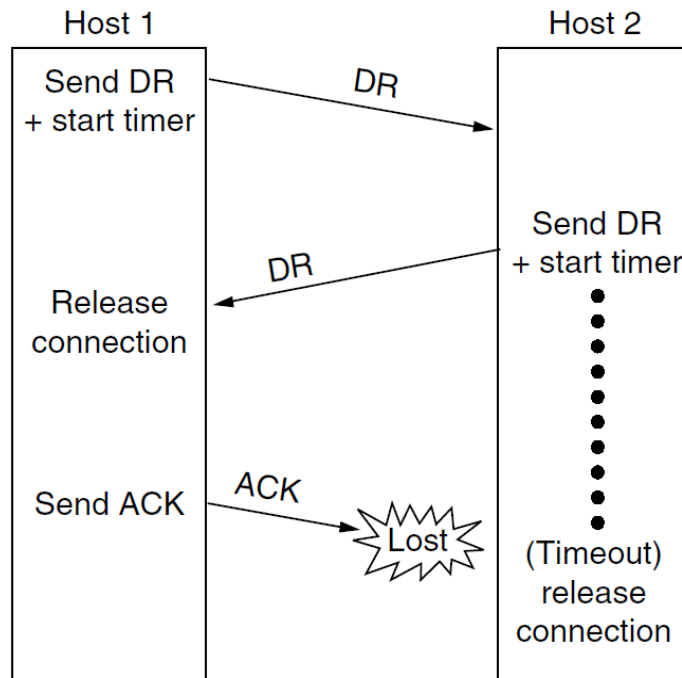
# Connection Release

- Strategies
  - ❑ 3-way handshake
  - ❑ Finite retry
  - ❑ Timeouts
- Normal release sequence, initiated by transport user on Host 1
  - ❑ DR=Disconnect Request
  - ❑ Both DRs are ACKed by the other side

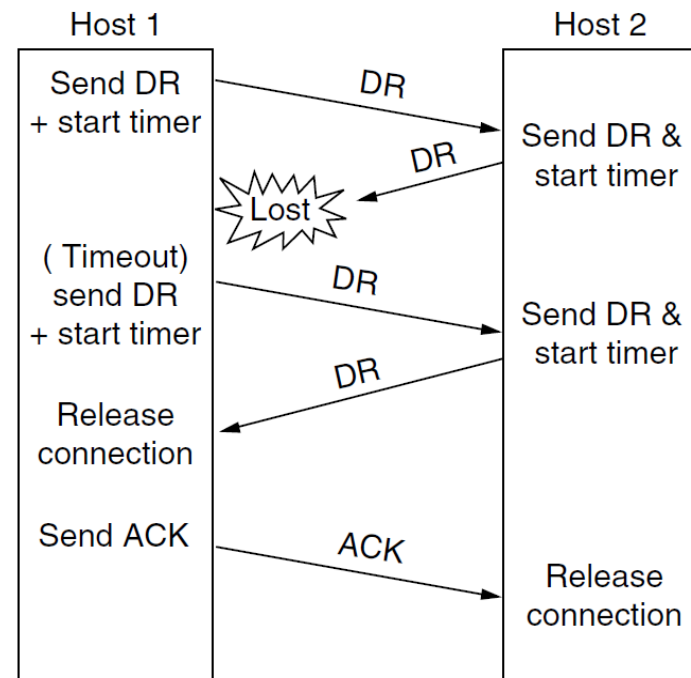


# Connection Release (Error Cases)

- Error cases are handled with timers and retransmission



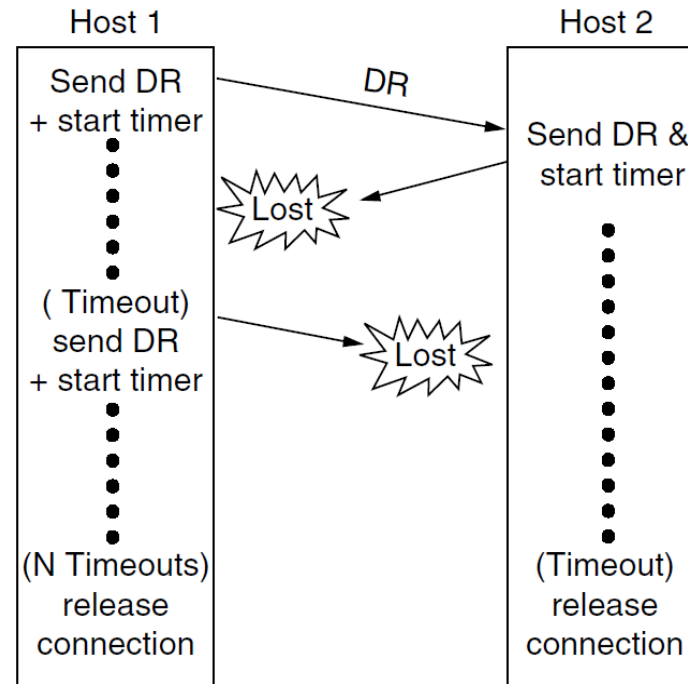
Final ACK lost,  
Host 2 times out



Lost DR causes  
retransmissions

# Connection Release (Error Cases)

- Error cases are handled with timers and retransmission



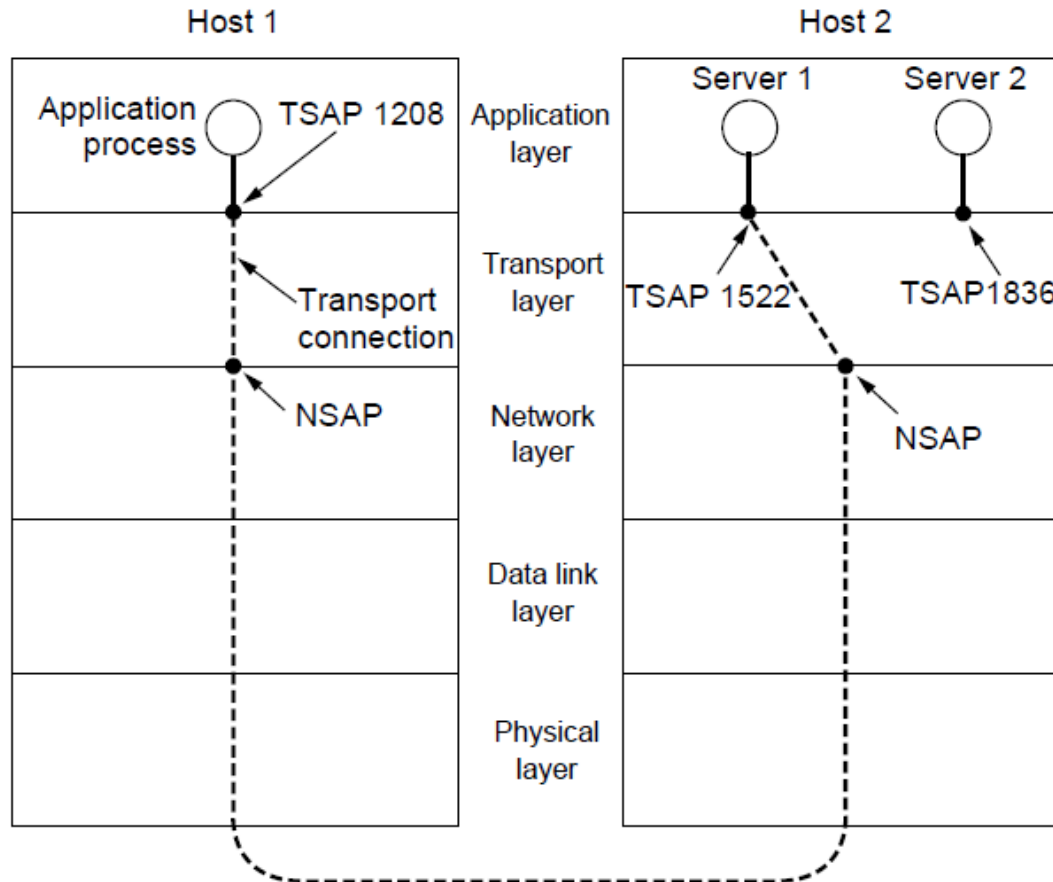
Extreme: Many lost DRs cause both hosts to timeout



# Addressing

- Specification of remote process to connect to is required at application and transport layers
- Addressing in the network layer is typically done using **Network Service Access Points (NSAPs)**
  - on the Internet, an NSAP is commonly interpreted as simply an **IP address**
- Addressing in transport layer is typically done using **Transport Service Access Points (TSAPs)**
  - on the Internet, a TSAP is commonly referred to as a **port** (e.g. port 80)

# TSAPs, NSAPs and Connections

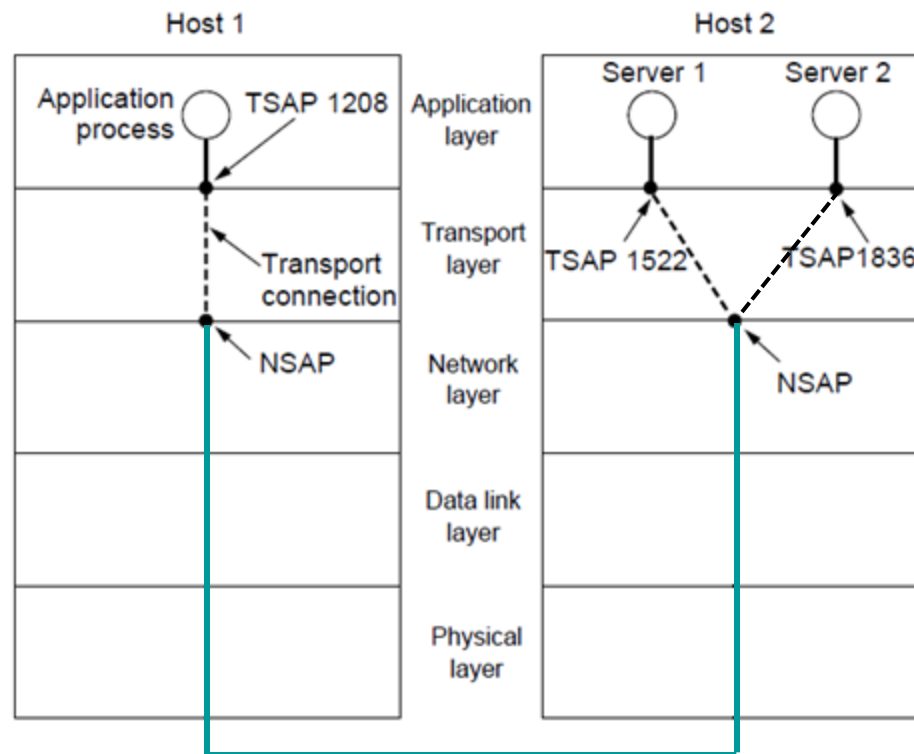


# Types of TSAP Allocation

- Static
  - Well-known services have standard allocated TSAPs/ports, which are embedded in OS
- Directory Assistance – Port-mapper
  - A new service must register itself with the portmapper, giving both its service name and TSAP
- Mediated
  - A process server intercepts inbound connections, launches requested server and attaches inbound connection, e.g. inetd server on UNIX systems

# Multiplexing

- Supports several conversations over the same network connection



# Outline

- Transport Layer Service
- Elements of Transport Protocols
- **Programming using Sockets**
- Transport Layer Protocols: TCP and UDP
- Quality of Service